PRESEARCH.md -- Shipyard Coding Agent

> *Completed before writing any code. All architecture decisions locked.*

Phase 1: Open Source Research

1. OpenCode (github.com/opencode-ai/opencode)

Source files read: `packages/opencode/src/session/prompt.ts` (main loop), `tool/edit.ts` (file editing), `session/compaction.ts` (context management), `tool/tool.ts` (tool registry), `permission/permission.ts` (permission gates)

File Editing -- Anchor-based string replacement:

- Primary tool: `edit(filePath, oldString, newString, replaceAll?)`
- `oldString` must appear exactly once in the file (uniqueness validation prevents accidental edits)
- File modification tracking via `ModTime()` -- read-before-write enforced
- Permission gates show unified diff preview before applying
- Fallback tools: `write()` for full rewrites, `patch()` for unified diffs
- LSP integration: 100ms post-edit delay for diagnostics, compiler errors fed back to LLM

Context management -- Automatic compaction + pruning:

- Overflow triggers at `context_limit - output_tokens - 20K buffer`
- Compaction agent generates structured summary (Goal, Instructions, Discoveries, Accomplished, Relevant Files)
- Pruning protects last ~40K tokens of tool output, strips older outputs while keeping execution records
- Token estimation: character-based ratio calculation (no tokenizer library)
- Media (images/PDFs) stripped during summarization, converted to text placeholders

Error handling:

- Known weakness: infinite loops on malformed JSON tool calls
- No loop breaker for repeated identical failures
- Permission denials propagate as tool errors to LLM for intelligent retry
- Feature requests exist for configurable retry (classify validation errors as non-retryable)

What I would take:

- Anchor-based `edit(old, new)` pattern -- proven most reliable for LLM-driven edits
- Compaction agent with structured summaries (not just truncation)
- Tool.define() pattern with Zod validation schemas
- 20+ built-in tools covering file I/O, search (ripgrep-backed), bash, web, delegation

What I would do differently:

- Add loop breaker for repeated failures (max 3 identical retries then escalate)
- Use cascading fallback for edits (exact match -> whitespace-normalized -> fuzzy -> full rewrite)
- Implement proper error classification (transient vs deterministic)

---

2. Open SWE / LangChain Open Engineer (github.com/langchain-ai/open-swe)

Note: The repo was renamed from "open-engineer" to "open-swe" and rebuilt on the Deep Agents framework.

Source studied: Architecture docs, blog posts, DeepWiki technical documentation

Architecture -- 4-agent LangGraph pipeline:

- **Manager Graph**: Entry point, message classification, session routing
- **Planner Graph**: Repository context gathering, execution plan generation, HITL approval gate
- **Programmer Graph**: Executes plans in sandbox, writes code, runs tests
- **Reviewer Graph**: Quality control, validates outputs, iterates with Programmer, opens PRs
- Parent-child coordination via LangGraph client API (`langGraphClient.runs.create()`)
- Built on LangGraph + Deep Agents framework (3-layer composition)

File editing -- Tool-based with line-range patches:

- Uses `edit_file` tool (line-based patching, not full rewrites)
- Workflow: `grep_raw` -> `view/read_file` -> `edit` -> `execute` (validate)
- Deep Agents built-in tools: `read_file` , `write_file` , `edit_file` , `ls` , `glob` , `grep`
- Context-based diff matching with cascading fallbacks (exact -> trimmed endings -> trimmed whitespace)

Context management -- Pre-hydrated + middleware-driven:

- Complete issue descriptions, Slack threads, PR comments injected at initialization
- Repository conventions from `AGENTS.md` injected into system prompt
- Middleware compresses/summarizes tool outputs before entering context window
- Large contexts offloaded to persistent storage via filesystem backend
- LangGraph checkpointing at every super-step (time travel debugging)

Multi-agent coordination:

- Subagent state fully isolated -- parent receives summary outputs only (no raw data)
- Custom subagents do NOT inherit tools by default (explicit `skills` parameter)
- Deterministic thread IDs (GitHub issue #, Slack thread ID) for message routing
- Concurrent messages queued per thread for sequential processing

Error handling -- Safety net middleware:

- `open_pr_if_needed` runs after agent completion (deterministic, not LLM-dependent)
- `ToolErrorMiddleware` catches exceptions, formats structured error messages
- Middleware cannot be skipped by agent routing decisions
- Sandbox restart between phases for clean state

What I would take:

- LangGraph state management with reducer annotations (`Annotated[list, add_messages]`)
- AGENTS.md convention injection pattern
- Safety net middleware (deterministic post-processing)
- Subagent state isolation (parent sees summaries only)
- ~14 curated tools ("curation over accumulation")

What I would do differently:

- Skip the 4-agent overhead for MVP -- single agent with tool calls is simpler
- Use anchor-based replacement instead of line-range (more robust to line drift)

- Add LangSmith tracing from day 1 (zero-config with env vars)

---

3. Claude Code (docs.anthropic.com/claude-code)

Source studied: Official docs (code.claude.com), Anthropic engineering blogs, Pragmatic Engineer deep dive

Agent loop -- Three-phase REPL:

- Gather context -> Take action -> Verify results
- Turn-based: `while(tool_call) -> execute tool -> feed results -> repeat`
- Loop terminates when Claude produces text without tool calls
- Minimal scaffolding philosophy: "Every model release, we delete a bunch of code"
- 90% of Claude Code's own codebase is written by itself

File editing -- str_replace_based_edit_tool:

- Commands: `view`, `str_replace` (anchor-based), `insert` (line number), `create`, `undo_edit`
- `str_replace`: requires exact match of `old_str`, replaces with `new_str`
- Must match whitespace, indentation, line endings exactly
- Line numbers included in view output for precise targeting
- Checkpoints before every edit (rewind capability)

Sub-agent coordination:

- Hierarchical: main session spawns specialized sub-agents with isolated contexts
- Agent types: Explore (read-only), General-purpose (all tools), Plan (read-only), Bash (CLI only)
- Foreground (blocking) vs Background (async with pre-approved permissions)
- No nesting: subagents cannot spawn other subagents
- Configuration via YAML frontmatter in `.claude/agents/`

Permission model:

- Read-only by default, asks before modifications
- 5 modes: Default, Auto-accept edits, Plan mode, Don't ask, Bypass permissions
- Sandboxing reduces permission prompts by 84%

Context management:

- Auto-compaction at ~95% capacity (configurable via env var)
- Clears older tool results first, then summarizes conversation
- Persistent rules go in CLAUDE.md (survives compaction)
- Subagent isolation: each gets fresh context window, returns summary only
- Tool search optimization: MCP tools deferred when >10% of context

What I would take:

- `str_replace(old_str, new_str)` as the primary edit mechanism
- Three-phase loop (gather -> act -> verify)
- Subagent isolation pattern (fresh context, summary return)
- Checkpoint-before-edit for safe rollback
- Minimal scaffolding philosophy

What I would do differently:

- Add `undo_edit` capability (Claude 4 removed it, I think it's valuable)

- Implement multi-file atomic edits (Claude Code edits one file at a time)
- Use LangGraph for state management instead of custom loop (better tracing)

---

4. Our Software Factory (Gauntlet/software-factory/)

Existing reusable code we already have:

Component	File	Reusable?
LLM Bridge	`core/runtime/llm.ts`	YES -- Multi-provider (Anthropic/OpenAI), 3-tier routing, token tracking
Agent Executor	`core/runtime/agent-executor.ts`	YES -- Role-based prompts, JSON output parsing, self-eval, CI runner
ACP Protocol	`core/protocol/acp.ts`	YES -- JSON-RPC 2.0 EventEmitter bus for inter-agent messaging
Athena Orchestrator	`core/orchestrator/athena.ts`	PARTIAL -- 7-phase pipeline, concurrent batch execution, dependency resolution
State Manager	`core/runtime/state.ts`	YES -- In-memory + file-persisted, event-driven, debounced writes
Daemon	`core/runtime/daemon.ts`	YES -- Express + WebSocket server, real-time broadcasting, preview management
Agent Templates	`agents/templates/*.md`	YES -- 9 role templates (athena, architect, engineer, ui-agent, qa, pm, repair, eval, reviewer)
Instinct8 Scoring	`core/scoring/instinct8.ts`	YES -- 3-dimension scoring (goal coherence, constraint recall, behavioral alignment)
Git Ops	`core/runtime/git-ops.ts`	YES -- Clone, pull, commit operations

Key patterns to port:

- Concurrent ticket execution with dependency graph resolution
- Self-eval -> QA -> PM decision loop (execute -> score -> ship/continue/escalate)
- Agent teardown after completion (memory conservation)
- Conventions injection from `shared/conventions.json`
- Learning persistence (cross-run knowledge)

What's missing for Shipyard:

- **Surgical file editing** -- factory uses full-file writes (`writeFileSync`), not surgical edits
- **Persistent agent loop** -- factory is batch-mode (decompose -> execute -> done), not interactive
- **Context injection** -- factory injects via prompt engineering, no runtime injection mechanism
- **Tracing** -- no LangSmith integration, only token counting

---

2. File Editing Strategy Decision

Chosen strategy: Anchor-based replacement (search and replace)

Justification:

- Used by Claude Code, OpenCode, Aider, Cline, RooCode -- proven in production at scale
- LLMs produce more reliable output in before/after format than diffs or line numbers
- No line-number drift problem (anchors are content-based, not position-based)
- No language-specific parser required (works on any file type)
- Simple to implement, simple to debug

Mechanism:

```
edit_file(path, old_string, new_string)
```

1. Read file contents
2. Search for `old_string` (exact match)
3. Verify uniqueness (must appear exactly once)
4. Replace with `new_string`
5. Write file back
6. Return success/failure + diff preview

Cascading fallback (4-tier):

1. Exact match
2. Whitespace-normalized match (trim leading/trailing per line)
3. Fuzzy match (Levenshtein distance < 10% of string length)
4. Full file rewrite (last resort, logged as degraded edit)

Failure modes and handling:

- **No match found**: Return error with closest match suggestion, agent retries with more context
- **Multiple matches**: Return error with match count and locations, agent provides more surrounding context
- **File changed between read and edit**: Re-read file, attempt match again, fail if still no match
- **Syntax error after edit**: Run linter/typecheck, feed errors back to agent for correction

---

Phase 2: Architecture Design

3. System Diagram

```
                        +------------------+
                        |   User (CLI/Web) |
                        +--------+---------+
                                 |
                        instruction / context injection
                                 |
                        +--------v---------+
                        |  Agent Server    |
                        |  (Express + WS)  |
                        +--------+---------+
                                 |
                  +-------------+-------------+
                  |                           |
          +--------v--------+        +--------v--------+
          | Persistent Loop |        | Context Store   |
          | (instruction    |        | (injected specs,|
          |  queue + state) |        |  schemas, test  |
```

```
          +--------+--------+        |  results)       |
                   |                  +--------+--------+
                   |                           |
          +--------v---------------------------v--------+
          |           LangGraph StateGraph               |
          |                                              |
          |  +--------+    +--------+    +--------+     |
          |  | Plan   |--->| Execute|--->| Verify |     |
          |  | Node   |    | Node   |    | Node   |     |
          |  +--------+    +---+----+    +---+----+     |
          |       ^             |             |          |
          |       |        tool calls    tool calls      |
          |       |             |             |          |
          |       |    +--------v--------+    |          |
          |       |    | Tool Router     |    |          |
          |       |    | - read_file     |    |          |
          |       |    | - edit_file     |    |          |
          |       |    | - write_file    |    |          |
          |       |    | - bash          |    |          |
          |       |    | - grep/glob     |    |          |
          |       |    | - spawn_agent   |    |          |
          |       |    +--------+--------+    |          |
          |       |             |             |          |
          |       +------error--+---success---+          |
          |                                              |
          |  Error Branch:                               |
          |  execute fails -> read error -> retry        |
          |  3 retries -> escalate to user               |
          +----------------------------------------------+
                   |
          +--------v--------+
          | LangSmith Trace |
          | (every node,    |
          |  every tool)    |
          +-----------------+
```

Node signatures (TypeScript contracts):

```typescript
// Shared state annotation (LangGraph)
interface ShipyardState {
  instruction: string;
  contexts: Array<{ label: string; content: string }>;
  steps: Array<{ index: number; description: string; files: string[]; status:
'pending' | 'in_progress' | 'done' | 'failed' }>;
  currentStepIndex: number;
  fileEdits: Array<{ file_path: string; tier: 1 | 2 | 3 | 4; old_string: string;
new_string: string; timestamp: number }>;
  toolCallHistory: Array<{ tool_name: string; tool_input: Record<string, unknown>;
tool_result: string; timestamp: number; duration_ms: number }>;
  messages: Array<{ role: 'assistant'; content: string }>;
  tokenUsage: { input: number; output: number };
  phase: 'planning' | 'executing' | 'verifying' | 'reviewing' | 'reporting' |
```

```
'error' | 'done';
  error: string | null;
  reviewVerdict: 'approve' | 'request_changes' | 'reject' | null;
  report: string | null;
}

// Node input/output contracts
type NodeFn = (state: ShipyardState) => Promise<Partial<ShipyardState>>;

// planNode: instruction + contexts → steps[] + phase='executing'
// executeNode: steps[currentStepIndex] → fileEdits[] + toolCallHistory[] +
phase='verifying'
// verifyNode: fileEdits[] → phase='reviewing' (pass) | phase='executing' (fail,
retry)
// reviewNode: fileEdits[] + messages[] → reviewVerdict + phase='reporting' |
'executing'
// reportNode: everything → report string + phase='done'
// errorRecoveryNode: error → phase='executing' (retry) | 'done' (escalate)
```

Edge routing (conditional):

```
// After execute: always verify
// After verify:
//   pass → review
//   fail + retries < 3 → execute (same step)
//   fail + retries >= 3 → error_recovery
// After review:
//   approve + more steps → execute (next step)
//   approve + no more steps → report
//   request_changes → execute (same step, with review feedback)
//   reject → error_recovery
// After error_recovery:
//   retry → plan (re-plan failed step)
//   escalate → report (with error summary)
```

4. File Editing Strategy (Step by Step)

1. **Agent reads file** via `read_file(path)` -- returns content with line numbers
2. **Agent identifies target** -- determines the block to change
3. **Agent calls** `edit_file(path, old_string, new_string)` : a. Server reads current file content b. Searches for `old_string` using 4-tier cascade (exact -> normalized -> fuzzy -> full) c. If unique match: replace, write, return diff d. If no match: return error + closest candidate e. If multiple matches: return error + match count
4. **Agent verifies** via `bash("npx tsc --noEmit")` or test command
5. **If verification fails**: agent reads error, calls edit_file again to fix

When location is wrong:

- Error message includes file content snippet around the best fuzzy match
- Agent re-reads file to update its mental model
- Agent retries with more surrounding context in `old_string`

- After 3 failed attempts: agent falls back to `write_file` (full rewrite) + logs degradation

Experimental: Hash-based line addressing (NOT PLANNED)

Research (Can Bölük, Feb 2026 — blog.can.ac) proposes hash-based editing where models reference lines by CRC32 content hashes instead of reproducing target text:

```
1:a3| function hello() {
2:f1|   return "world";
3:0e| }
```

Benchmark results (Can Bölük, 15 models × 180 React tasks):

Model	str_replace/patch	Hashline	Delta
Grok Code Fast	6.7%	68.3%	+916%
Most models	baseline	+5-14 pts	-20% output tokens

Counter-evidence (why we're NOT adopting this):

- **Anthropic rejected it**: Claude Code feature request #25775 closed NOT_PLANNED (Mar 2026)
- **Python penalty**: DEV.to counter-benchmark shows 95% replace vs 70% hashline for Gemini-3-flash on Python
- **Training data bias**: Models are trained on str_replace patterns; hashline fights this
- **Distraction hypothesis**: Random hash prefixes on every line may reduce reasoning quality
- **Quality signal loss**: Requiring models to reproduce target text acts as a verification step ("think before you answer")
- **Zero production deployments**: No major IDE or agent framework has shipped this (as of Mar 2026)
- **Biggest gains on weakest models**: +916% Grok Fast = terrible → mediocre. For Opus/Sonnet, marginal improvement

Verdict: Monitor but don't build. The 4-tier cascade (exact → whitespace → fuzzy → full rewrite) handles Opus/Sonnet well. Revisit only if we target cheap/weak models for cost optimization.

5. Multi-Agent Design

Orchestration model: Supervisor pattern via LangGraph

```
                    +-------------------+
                    |   Supervisor      |
                    |   (main agent)    |
                    +---------+---------+
                              |
            +---------------+---------------+
            |               |               |
     +---------v---+  +------v------+  +----v--------+
     | Worker A    |  | Worker B    |  | Worker C    |
     | (frontend)  |  | (backend)   |  | (tests)     |
     +-------------+  +-------------+  +-------------+
```

Typed contracts:

```typescript
// Subtask definition (supervisor decomposes instruction into these)
interface Subtask {
  id: string;
  description: string;
  files: string[];               // expected files to touch
  dependencies: string[];        // subtask IDs that must complete first
  workerType: 'frontend' | 'backend' | 'tests' | 'general';
}

// Worker result (returned to supervisor)
interface WorkerResult {
  subtaskId: string;
  status: 'success' | 'failed';
  fileEdits: FileEdit[];         // what files were changed
  fileHashes: Map<string, string>; // post-edit SHA256 per file (for conflict
detection)
  summary: string;               // natural language summary for supervisor context
  errors: string[];              // any unresolved errors
}

// Conflict detection
interface ConflictReport {
  file_path: string;
  workerA: string;               // subtask ID
  workerB: string;
  type: 'non_overlapping' | 'overlapping' | 'structural';
  resolution: 'auto_merge' | 'supervisor_replan' | 'type_check';
}
```

Parallel dispatch via LangGraph `Send()`:

```typescript
import { Send } from '@langchain/langgraph';

// Supervisor node returns Send() commands for parallel execution
function supervisorNode(state: ShipyardState): Command {
  const subtasks = decomposeInstruction(state.instruction);

  // Independent subtasks run in parallel
  const independent = subtasks.filter(t => t.dependencies.length === 0);
  return new Command({
    goto: independent.map(t => new Send('worker_node', {
      ...state,
      currentSubtask: t,
      isolatedContext: true,  // fresh context window
    })),
  });
}
```

Conflict resolution state machine:

```
Worker A done + Worker B done
    ↓
Compare fileHashes (files both workers touched)
    ↓
No shared files → merge all edits → continue
    ↓
Shared files, non-overlapping edits → sequential re-apply → type-check
    ↓
Shared files, overlapping edits → supervisor re-plans conflicting subtask
    ↓
Type-check fails after merge → feed errors to responsible worker → retry
```

Alternative: Git worktree isolation. Claude Code and Cursor use isolated git worktrees for multi-agent work — each worker gets its own worktree, edits independently, supervisor merges via `git merge`. More robust than file hash comparison but heavier setup. Deferred to Phase 6 alongside parallel execution.

MVP approach: Sequential execution (no parallel workers). Conflict resolution is Phase 6. The contracts above are designed so the agent can add parallelism later without restructuring state.

6. Context Injection Spec

Types of context:

Context Type	Format	Injection Point	Max Size
Spec/PRD	Markdown string	System prompt (pre-loaded)	10K tokens
Schema (DB/API)	SQL DDL or TypeScript types	System prompt or tool result	5K tokens
Previous output	File contents or summary	State (conversation history)	Unlimited (compacted)
Test results	Stdout/stderr text	Tool result (after bash execution)	5K tokens (truncated)
Codebase conventions	AGENTS.md / CLAUDE.md content	System prompt (always present)	3K tokens
Runtime user message	Free text	User message (mid-loop injection)	2K tokens

Context type definition:

```typescript
interface ContextBlock {
  label: string;           // displayed as markdown header
  content: string;         // the actual context text
  source: 'spec' | 'schema' | 'conventions' | 'user' | 'tool_result';
  priority: number;        // 0 = always keep, 1 = keep if room, 2 = compactable
  injectedAt: number;      // timestamp
}
```

System prompt template (actual template the agent populates):

```
const SYSTEM_PROMPT_TEMPLATE = `You are Shipyard, an autonomous coding agent.

# Codebase Conventions
{{conventions}}

# Current Context
{{#each contexts}}
## {{this.label}}
{{this.content}}

{{/each}}

# Rules
- Read files before editing (understand before modifying)
- Use edit_file for surgical changes (preferred over write_file)
- Use write_file only for new files
- Run verification after every edit (bash: tsc --noEmit, vitest)
- When done with a step, say "STEP_COMPLETE"
- If stuck after 3 attempts, say "STUCK: <reason>"`;
```

Injection mechanism:

- **At loop start**: System prompt includes persistent context (specs, conventions, schemas) via template above
- **Mid-loop**: User can inject new context via WebSocket message — server appends to conversation as a new user message with `source: 'user'`
- **Post-tool**: Tool results automatically injected (file contents, command output, test results)
- **Cross-turn**: LangGraph checkpointing preserves full conversation state
- **Compaction**: When context exceeds 80% of window, drop `priority >= 2` blocks first, then summarize `priority 1` blocks

Format: All context injected as plain text with markdown headers for structure. No JSON wrapping for context (LLMs handle markdown better than structured formats for comprehension).

Test vectors:

Input	Expected Behavior
Inject spec with `priority: 0`	Always present in system prompt, survives compaction
Inject 200K tokens of tool results	Compaction triggers, oldest `priority 2` blocks dropped first
User sends WS message mid-edit	Appended as user message, agent processes on next turn
Same label injected twice	Second replaces first (deduped by label)

7. Additional Tools

Tool	Input	Return	Notes

`read_file(path, offset?, limit?)`	`{ file_path: string, offset?: number, limit?: number }`	`{ success: true, content: string, lines: number, truncated: boolean }`	Line numbers prepended. Default limit: 2000 lines.
`edit_file(path, old, new)`	`{ file_path: string, old_string: string, new_string: string }`	`{ success: boolean, tier: 1|2|3|4, diff_preview: string, message: string }`	4-tier cascade. Tier in result tells agent which fallback matched.
`write_file(path, content)`	`{ file_path: string, content: string }`	`{ success: true, bytes_written: number }`	Creates parent dirs. Use only for new files.
`bash(command, timeout?, cwd?)`	`{ command: string, timeout?: number, cwd?: string }`	`{ success: boolean, stdout: string, stderr: string, exit_code: number }`	Default timeout 30s, max 120s. Output truncated at 50K chars.
`grep(pattern, path?, glob?)`	`{ pattern: string, path?: string, glob?: string, max_results?: number }`	`{ success: true, matches: Array<{ file: string, line: number, text: string }>, total: number }`	Ripgrep-backed. Default max 50 results.
`glob(pattern, cwd?)`	`{ pattern: string, cwd?: string }`	`{ success: true, files: string[], total: number }`	Sorted by modification time.
`ls(path)`	`{ path: string }`	`{ success: true, entries: Array<{ name: string, type: 'file'|'dir', size: number }> }`	Includes type and byte size.
`spawn_agent(task, tools?)`	`{ task: string, tools?: string[] }`	`{ success: boolean, summary: string, fileEdits: FileEdit[] }`	Isolated context. Returns summary only. MVP: not implemented (sequential).
`inject_context(text)`	`{ label: string, content: string, priority?: number }`	`{ success: true }`	Adds to state.contexts. Deduped by label.
`ask_user(question)`	`{ question: string }`	`{ answer: string }`	Pauses loop via LangGraph `interrupt()`. Resumes on user response.

Tool dispatch pattern (hooks + overlay):

```
// All tools dispatched through a single function with interception hooks
async function dispatchTool(
```

```
  name: string,
  input: Record<string, unknown>,
  hooks?: ToolHooks,    // before/after recording, logging, validation
  overlay?: FileOverlay, // copy-on-write snapshots for rollback
): Promise<Record<string, unknown>>;

// Hooks record FileEdit[] and ToolCallRecord[] automatically
// Overlay snapshots files before mutation (edit_file, write_file)
// Rollback: overlay.rollbackAll() reverts all edits in current step
```

Phase 3: Stack and Operations

8. Framework Choice

LangGraph (TypeScript) + Anthropic SDK — authenticated via Claude Max plan

Why:

- LangGraph gives us state management, checkpointing, conditional edges, and tracing out of the box
- Claude Max plan gives flat-rate access to Opus 4.6 + Sonnet 4.5 — no per-token billing
- TypeScript matches our existing codebase (Ship, software factory)
- LangSmith tracing is automatic with env vars (zero code changes)
- `StateGraph` pattern maps cleanly to plan → execute → verify → review → report loop
- Our software factory already uses Anthropic SDK

Why not alternatives:

- **Custom loop (no framework)**: No built-in persistence, checkpointing, or tracing. We'd rebuild what LangGraph provides.
- **Python/FastAPI**: Stack mismatch with Ship (TypeScript), slower iteration.
- **LangChain alone**: No graph-based state management, harder multi-agent.

Authentication: Claude Max plan API key (subscription-based, not usage-based). All models (Opus, Sonnet) included in the flat rate.

Dependencies (exact versions):

```
{
  "@langchain/langgraph": "^0.2.63",
  "@langchain/core": "^0.3.44",
  "@anthropic-ai/sdk": "^0.52.0",
  "zod": "^3.24.2"
}
```

Graph wiring skeleton:

```
import { StateGraph, Annotation, END } from '@langchain/langgraph';

const ShipyardAnnotation = Annotation.Root({
  phase: Annotation<string>(),
  steps: Annotation<Step[]>(),
```

```
  currentStepIndex: Annotation<number>(),
  // ... (full state in Section 3)
});

const graph = new StateGraph(ShipyardAnnotation)
  .addNode('plan', planNode)
  .addNode('execute', executeNode)
  .addNode('verify', verifyNode)
  .addNode('review', reviewNode)
  .addNode('error_recovery', errorRecoveryNode)
  .addNode('report', reportNode)
  .addEdge('__start__', 'plan')
  .addEdge('plan', 'execute')
  .addEdge('execute', 'verify')
  .addConditionalEdges('verify', routeAfterVerify)    // pass→review,
fail→execute/error
  .addConditionalEdges('review', routeAfterReview)     // approve→execute/report,
changes→execute
  .addConditionalEdges('error_recovery', routeAfterError) // retry→plan,
escalate→report
  .addEdge('report', END);

const app = graph.compile({ checkpointer: new MemorySaver() });
```

Model selection per node:

```
// config/model-policy.ts
const MODEL_CONFIGS = {
  planning:     { model: 'claude-opus-4-6',             maxTokens: 4096,
temperature: 0.3 },
  coding:       { model: 'claude-sonnet-4-5-20250929',  maxTokens: 8192,
temperature: 0.2 },
  review:       { model: 'claude-opus-4-6',             maxTokens: 2048,
temperature: 0.2 },
  verification: { model: 'claude-sonnet-4-5-20250929',  maxTokens: 2048,
temperature: 0.0 },
  summary:      { model: 'claude-sonnet-4-5-20250929',  maxTokens: 2048,
temperature: 0.3 },
};
```

Acceptance criteria:

- `graph.compile()` succeeds without errors
- `app.invoke({ instruction: "..." })` runs through plan → execute → verify → review → report
- LangSmith trace URL returned in final state
- Checkpointer persists state between invocations (thread_id)
- Token usage tracked per node via ConsumptionTracker

8b. Provider Strategy

Decision: Claude-only via Max plan. No multi-provider abstraction needed for MVP.

Shipyard uses the Anthropic SDK directly, authenticated via Claude Max plan API key. All tiers (planning, execution, review, verification, summary) use Claude models. No third-party models.

Coupling is intentional, not a problem:

- Single `new Anthropic()` client, shared across all phases
- `config/model-policy.ts` centralizes model selection (Opus for planning/review, Sonnet for execution)
- Tool dispatch (`tools/index.ts:dispatchTool`) operates on plain objects — zero provider coupling
- If we ever need a second provider, build a thin `LLMProvider` interface (~200 LOC) at that point

Industry context (2026):

- **MCP** (Model Context Protocol): Open standard for tool schemas, governed by Linux Foundation. Our tool schemas are already JSON Schema — small leap to MCP for tool extensibility.
- **Agent Skills**: Anthropic + OpenAI standardizing portable skill definitions. Skills work across providers.
- Provider portability is a post-MVP concern. Tool handlers are already portable; only the LLM call layer is Claude-specific.

9. Persistent Loop

Where: Express + WebSocket server (same pattern as our software factory daemon)

REST API routes:

Method	Path	Request	Response	Description
POST	`/instruct`	`{ instruction: string, contexts?: ContextBlock[] }`	`{ runId: string }`	Queue new instruction
POST	`/respond`	`{ runId: string, answer: string }`	`{ ok: true }`	Resume after `ask_user` interrupt
GET	`/status`	—	`{ phase: string, currentStep: number, totalSteps: number }`	Current run state
GET	`/runs/:id`	—	`ShipyardState`	Full state for a run
GET	`/runs/:id/report`	—	`{ report: string, tokenUsage, traceUrl }`	Final report
DELETE	`/runs/:id`	—	`{ ok: true }`	Cancel a run

WebSocket message contract:

```
// Client → Server
type ClientMessage =
  | { type: 'instruct'; instruction: string; contexts?: ContextBlock[] }
  | { type: 'respond'; runId: string; answer: string }
```

```
  | { type: 'cancel'; runId: string };

// Server → Client
type ServerMessage =
  | { type: 'phase_change'; runId: string; phase: string; step?: number }
  | { type: 'tool_call'; runId: string; tool: string; input: Record<string, unknown>
}
  | { type: 'tool_result'; runId: string; tool: string; success: boolean;
duration_ms: number }
  | { type: 'ask_user'; runId: string; question: string }
  | { type: 'report'; runId: string; report: string; traceUrl?: string }
  | { type: 'error'; runId: string; message: string };
```

Lifecycle:

- Express server runs continuously on `localhost:4200`
- Instructions arrive via REST POST or WebSocket message → queued in FIFO `InstructionQueue`
- Agent loop processes one instruction at a time (no parallel runs in MVP)
- State persisted to disk (JSON) with debounced writes (200ms)
- Graceful shutdown saves state on SIGTERM/SIGINT
- On restart: loads last state from disk, resumes if interrupted mid-run

Not fire-and-forget: Server maintains conversation state between instructions. New instruction appends to existing conversation, agent continues with full context.

Context compaction mechanism (when budget check returns 'compact'):

Shipyard uses structured compaction (OpenCode pattern), NOT naive truncation. Naive truncation loses 60%+ of useful context.

```
// Compaction agent generates a structured summary
const COMPACTION_PROMPT = `Summarize the conversation so far into this structure:
## Goal
<what the user wants to achieve>
## Instructions
<any constraints, preferences, or instructions from the user>
## Discoveries
<what we learned about the codebase — file locations, patterns, types>
## Accomplished
<what steps are done, what files were edited>
## Relevant Files
<files still relevant to remaining work — path + 1-line description>`;

// Process:
// 1. Budget check returns 'compact'
// 2. Send full conversation to Sonnet with COMPACTION_PROMPT
// 3. Replace all messages with single summary message
// 4. Keep priority-0 context blocks intact (conventions, schemas)
// 5. Drop priority-2 context blocks (old tool results)
// 6. Continue with compacted context (~20-30% of original size)
```

Acceptance criteria:

- Server starts, accepts instruction, returns `runId`
- WebSocket broadcasts `phase_change` events in real-time
- `ask_user` pauses loop, `respond` resumes it
- Server survives restart without losing in-progress state

10. Token Budget and Consumption Tracking

Billing model: Claude Max plan (flat-rate subscription). No per-token charges. Token tracking exists for consumption analytics, rate limit awareness, context management, and latency optimization — not cost control.

Per invocation budget: ~8K output tokens, ~200K context window (2026 models)

Consumption estimates (for analytics, not billing):

Scenario	Input tokens	Output tokens	Notes
Single edit cycle	~5K	~2K	1 plan + 1 edit + 1 verify
Feature (10 cycles)	~50K	~20K	Typical feature implementation
Module (50 cycles)	~250K	~100K	Large refactor or new module
Full rebuild (500+ cycles)	~2.5M	~1M	Includes retries + review passes

These numbers matter for: (1) context window management, (2) rate limit headroom, (3) understanding agent efficiency over time.

Budget enforcement mechanism:

```
interface TokenBudget {
  maxInputPerRun: number;     // default: 500_000 (context management, not cost)
  maxOutputPerRun: number;    // default: 100_000
  compactionThreshold: number; // 0.8 = compact at 80% of context window
  hardStopThreshold: number;  // 0.95 = abort if 95% consumed (prevents infinite
loops)
}

// Checked after every LLM call:
function checkBudget(usage: TokenUsage, budget: TokenBudget): 'ok' | 'compact' |
'abort' {
  const inputRatio = usage.input / budget.maxInputPerRun;
  if (inputRatio >= budget.hardStopThreshold) return 'abort';
  if (inputRatio >= budget.compactionThreshold) return 'compact';
  return 'ok';
}
// 'compact' → trigger context compaction before next LLM call
// 'abort' → mark run as failed with budget_exceeded error (runaway prevention)
```

Consumption instrumentation (per-run tracking):

```
// Accumulated in state after every LLM call
interface ConsumptionTracker {
```

```
  totalInputTokens: number;
  totalOutputTokens: number;
  cacheCreationTokens: number;   // from response.usage.cache_creation_input_tokens
  cacheReadTokens: number;       // from response.usage.cache_read_input_tokens
  cacheHitRate: number;          // cacheRead / (cacheRead + cacheCreation +
uncached)
  perPhaseBreakdown: Map<string, { input: number; output: number; calls: number;
avgLatencyMs: number }>;
}

// Reported in final state + trace
// Agent: "Run complete. Tokens: 12.4K in / 3.2K out. Cache hit: 84%. Avg latency:
1.2s/call."
```

Efficiency strategies:

- Opus for planning/review only; Sonnet for execution (Opus is slower, use only where reasoning depth matters)
- Prefix caching (see Gap Analysis): 80%+ target — reduces latency and rate limit pressure
- Context compaction at 80% capacity (structured summary, not truncation — see Section 9)
- Sub-agent isolation prevents context bloat
- Budget hard-stop prevents runaway loops from consuming the full context window

11. Bad Edit Recovery

Layered verification pipeline (run after every edit):

1. **Lint** — `bash("npx eslint --no-warn-ignored <file>")` — style/formatting (fast, <2s)
2. **Type-check** — `bash("npx tsc --noEmit")` — structural correctness (medium, 5-15s)
3. **Unit tests** — `bash("npx vitest run <related_test>")` — behavioral correctness (slow, 10-30s)
4. **AI review** — review node (Opus) — semantic correctness (LLM call, 5-10s)
5. **Human gate** — `ask_user` interrupt — for high-risk changes (configurable)

Error classification + handling:

```
type ErrorClass = 'transient' | 'deterministic' | 'fatal';

function classifyError(error: string, tool: string): ErrorClass {
  // Transient: retry with backoff
  if (/ECONNRESET|ETIMEDOUT|rate.limit|429|503|overloaded/.test(error)) return
'transient';

  // Fatal: escalate immediately, don't waste retries
  if (/EACCES|EPERM|ENOSPC|permission denied|out of memory/.test(error)) return
'fatal';

  // Deterministic: re-read context, retry with different approach
  return 'deterministic';
}
```

Error Class	Max Retries	Strategy	Example
Transient	3	Exponential backoff (1s, 2s, 4s), same input	API timeout, rate limit, 503
Deterministic	3	Re-read file, retry with more context in old_string	"String not found", type error after edit
Fatal	0	Rollback via FileOverlay, escalate to user/supervisor	Permission denied, disk full

Decision tree:

```
Edit applied
  ↓
Run lint → fail? → classify error → handle per class
  ↓ pass
Run type-check → fail? → feed errors to agent → agent calls edit_file to fix
  ↓ pass                    ↓ retry 3x failed
Run tests → fail? →        → rollback via overlay.rollbackAll()
  ↓ pass                    → escalate to error_recovery node
AI review (Opus) → request_changes? → feed review to agent → re-edit
  ↓ approve
Continue to next step
```

Rollback mechanism: FileOverlay (copy-on-write) snapshots every file before first edit in a step. On verification failure after max retries, `overlay.rollbackAll()` restores all files to pre-step state. On success, `overlay.commit()` discards snapshots.

Git safety net: All edits happen on a branch; `git diff` shows all changes for review.

Test vectors:

Scenario	Expected Behavior
edit_file succeeds, tsc fails	Agent reads error, calls edit_file again (deterministic retry)
edit_file succeeds, tsc fails 3x	Rollback all files in step, move to error_recovery
API returns 429	Wait 1s, retry same call (transient)
File permission denied	Immediate escalate, no retries (fatal)
Vitest passes, Opus review says "request_changes"	Feed review feedback, agent re-edits

12. Security and Sandboxing

Threat model: Shipyard executes LLM-generated bash commands and writes files to disk. Without isolation, a hallucinated `rm -rf /` or a malicious instruction could destroy the host filesystem.

Isolation strategy (layered):

Layer	Mechanism	What it prevents
Tool allowlist	Only 7 tools exposed (read, edit, write, bash, grep, glob, ls). No network, no process management.	Arbitrary system access
Bash command filtering	Pre-execution regex filter blocks destructive patterns (`rm -rf`, `sudo`, `chmod 777`, `curl | sh`, `mkfs`, `dd`)	Accidental/hallucinated destruction
Working directory jail	All file operations scoped to project root. Paths resolved + validated: reject `../` traversal, symlink escapes, absolute paths outside project	Filesystem escape
FileOverlay snapshots	Copy-on-write before every edit. `rollbackAll()` on failure.	Irreversible file corruption
Git safety net	All edits on a branch. `git diff` before any commit. Agent cannot force-push or delete branches.	Unreviewed changes reaching main
Token budget hard-stop	Abort run at 95% budget consumed	Infinite loop / runaway execution

Bash tool constraints:

```
interface BashConstraints {
  timeout: number;          // default: 30s, max: 120s
  maxOutputBytes: number;   // 50KB — truncate beyond
  blockedPatterns: RegExp[]; // rm -rf, sudo, etc.
  allowedCwd: string;       // must be under project root
  networkAccess: false;     // no curl, wget, fetch in bash
}
```

MVP approach: Software-level isolation (allowlist + filtering + overlay). Docker containerization is a Phase 6 enhancement for multi-tenant use. For single-user local execution (our case), the layers above are sufficient and match Claude Code's own sandboxing model.

What the agent cannot do:

- Access files outside the project directory
- Execute network requests from bash
- Install system packages
- Modify git config or force-push
- Run commands longer than 120 seconds
- Write more than 50KB of output per command

13. Trace Logging

What gets logged (via LangSmith):

- Every LLM call: input messages, output, token counts, latency
- Every tool call: tool name, arguments, result, duration
- Every node transition: from-node, to-node, state snapshot
- Every error: error type, stack trace, recovery action

Complete run trace for a typical edit:

```
[Trace: edit-login-component]
  1. plan_node
     - LLM call: "Read the login component and add email validation"
     - Tool: read_file("src/components/Login.tsx") -> 150 lines
     - Tool: grep("email.*valid", "src/") -> 2 matches
     - Decision: edit Login.tsx, add Zod validation
  2. execute_node
     - Tool: edit_file("src/components/Login.tsx", old_block, new_block) -> success
     - Tool: bash("npx tsc --noEmit") -> 0 errors
  3. verify_node
     - Tool: bash("npx vitest run Login.test") -> 5/5 pass
     - LLM call: "Verification complete, all tests pass"
     - -> END (success)

  Tokens: 3,200 input / 890 output
  Cache hit: 87% (system prompt cached)
  Duration: 8.2s
  Trace URL: https://smith.langchain.com/o/xxx/projects/p/xxx/r/xxx
```

14. Rate Limit Design (Max Plan)

Claude Max plan throughput constraints (not cost constraints):

Limit	Value	Impact
Requests per minute	Varies by tier	Burst tool-use loops can hit this
Tokens per minute (input)	Varies by tier	Large context sends (full module reads) can throttle
Tokens per minute (output)	Varies by tier	Review passes with long outputs
Concurrent requests	Limited	Affects multi-agent parallelism

Backpressure handling:

```typescript
interface RateLimitStrategy {
  // Retry with exponential backoff on 429
  maxRetries: 3;
  baseDelayMs: 1000;        // 1s, 2s, 4s

  // Proactive throttling (prevent hitting limits)
  minDelayBetweenCallsMs: 200;  // 200ms gap between API calls
  batchToolResults: true;       // combine multiple tool results into single message
}

// Rate limit headers from API response:
// x-ratelimit-limit-requests, x-ratelimit-remaining-requests
// x-ratelimit-limit-tokens, x-ratelimit-remaining-tokens
// Track these in ConsumptionTracker for proactive throttling
```

Multi-agent implications:

- MVP: Sequential execution (single agent), rate limits are not a concern
- Phase 6 (parallel workers): Each worker shares the same API key → shared rate limit pool
- Strategy: Supervisor tracks remaining quota, throttles worker dispatch when < 20% remaining

15. Evaluation and Success Metrics

How we know the agent works:

Metric	Target	Measurement
Edit success rate (tier 1-2)	90%+	`tier <= 2` / total edits
Edit success rate (any tier)	98%+	successful edits / total attempts
Verification pass rate (first try)	70%+	first-try pass / total verify calls
Verification pass rate (with retries)	95%+	eventual pass / total verify calls
Full task completion	80%+	tasks reaching `report` phase without escalation
Average task latency	< 60s for single-file edits	wall-clock from instruction to report

Test methodology:

1. **Smoke tests** (automated, run on every change):
    - Feed 5 canonical instructions (add function, fix bug, refactor, add test, multi-file change)
    - Assert: edit applied, typecheck passes, test passes, report generated
    - Assert: token counts within expected range (no runaway)
2. **Regression suite** (weekly):
    - 20 tasks of increasing complexity
    - Track success rate, token consumption, latency over time
    - Compare against previous week's baseline
3. **Failure analysis**:
    - Every escalation logged with full trace URL
    - Weekly review: categorize failures (bad plan, wrong file, edit drift, test flake)
    - Feed failure patterns back into system prompt refinement

Not running SWE-bench — our codebase (Ship, 101K LOC) is the benchmark. Success = Shipyard can implement real Ship features without human intervention.

Existing Code Reuse Plan

From `software-factory/`:

What	Source	Adaptation Needed

LLM Bridge	`core/runtime/llm.ts`	Port directly, add LangSmith callback
ACP Bus	`core/protocol/acp.ts`	Use as-is for inter-agent messaging
State Manager	`core/runtime/state.ts`	Simplify (remove ticket/project specifics)
Daemon Server	`core/runtime/daemon.ts`	Strip factory-specific routes, add agent loop routes
Agent Templates	`agents/templates/*.md`	Create new coding-agent-specific templates
Instinct8 Scoring	`core/scoring/instinct8.ts`	Use for self-eval quality gates
Git Ops	`core/runtime/git-ops.ts`	Use for branch management during edits

New code needed:

- `edit_file` tool with 4-tier cascading fallback
- LangGraph StateGraph wiring (plan → execute → verify nodes)
- LangSmith tracing integration
- Context injection middleware
- Sub-agent spawning via LangGraph `Send()`
- Security layer (bash filtering, path validation, FileOverlay)

2026 Gap Analysis

Cost Model: Max Plan Economics

Shipyard runs on Claude Max plan (flat-rate subscription). No per-token billing. This eliminates cost-per-run as a design constraint — the agent can retry freely, use Opus for review, and run verification loops without budget anxiety.

Why token tracking still matters:

1. **Rate limits** — Max plan has throughput limits (requests/min, tokens/min). Heavy runs can hit them. See Section 10 for rate limit design.
2. **Context management** — A 500-cycle rebuild generates ~2.5M input tokens. Without compaction, context overflows.
3. **Efficiency analytics** — Tracking tokens per phase reveals optimization opportunities (which phases are bloated? where is the agent spinning?).
4. **Latency** — More tokens = slower responses. Prefix caching reduces latency even when cost is flat.

Consumption reference (for context management, not billing):

Scenario	Input tokens	Output tokens	Notes
Single edit cycle	~5K	~2K	Baseline
Feature (10 cycles)	~50K	~20K	Typical task
Module (50 cycles)	~250K	~100K	Needs compaction mid-run
Full rebuild (500+ cycles)	~2.5M	~1M	Multiple compaction + sub-agent isolation required

Context Window (2026 State)

As of 2026, context windows are:

- Claude Opus 4.6: 200K input tokens
- Claude Sonnet 4.6: 200K input tokens
- GPT-5.3: 256K input tokens

The original doc assumed 100K. With 200K, a single context window can hold ~800-1000 files of typical size (200 lines avg). This means the agent can reason about ~60% of Ship's codebase in a single pass without compaction.

Implications for Shipyard:

- Planning node can ingest entire module context (all files in a directory tree) without chunking
- Verification node can hold full type-check error output even for large projects
- Compaction is still needed for multi-step runs (10+ edit cycles accumulate tool results fast)
- Sub-agent isolation remains valuable for parallelism, not just context management

Prefix Caching Design (Latency + Throughput Optimization)

Claude Code achieves 92% prefix reuse rate. On Max plan, prefix caching matters for **latency** (cached prefixes process faster) and **rate limit headroom** (cached tokens count less against throughput limits), not cost.

System prompt structure for maximum cache hits:

```
┌──────────────────────────────────────────────────────────┐
│ STATIC PREFIX (cached across all LLM calls)              │  ← cached
│ ┌──────────────────────────────────────────────────────┐ │
│ │ System instruction (EXECUTE_SYSTEM / PLAN_SYSTEM)    │ │
│ │ Tool definitions (TOOL_SCHEMAS — 7 tools, ~2K tok)   │ │
│ │ Codebase conventions (CLAUDE.md / AGENTS.md)         │ │
│ │ DB schema (if loaded)                                │ │
│ └──────────────────────────────────────────────────────┘ │
│ ≈10K tokens — identical across all calls in a run        │
├──────────────────────────────────────────────────────────┤
│ DYNAMIC SUFFIX (changes per call)                        │  ← NOT cached
│ ┌──────────────────────────────────────────────────────┐ │
│ │ Current step description                             │ │
│ │ Injected contexts (spec, test results)               │ │
│ │ Conversation history (tool calls + results)          │ │
│ └──────────────────────────────────────────────────────┘ │
└──────────────────────────────────────────────────────────┘
```

Implementation:

```
// Anthropic SDK: cache_control marks the prefix boundary
const systemMessages = [
  {
    type: 'text',
    text: STATIC_SYSTEM_PROMPT,      // instructions + conventions + schemas
    cache_control: { type: 'ephemeral' },  // ← tells Claude to cache this block
  },
```

```
  {
    type: 'text',
    text: dynamicContext,             // step-specific context (NOT cached)
  },
];
```

Targets:

- Cache hit rate: **80%+** (Claude Code achieves 92%)
- Measurement: Track `cache_creation_input_tokens` and `cache_read_input_tokens` from API response usage
- Reuse window: 5-minute TTL, auto-refreshed on subsequent calls within same run

Impact:

- **Latency**: Cached prefixes skip re-processing — faster time-to-first-token on every call after the first
- **Rate limits**: Cached tokens consume less throughput quota — more headroom for multi-step runs
- **Context efficiency**: Static 10K prefix amortized across all calls in a run

MCP (Standard 2026 Practice)

MCP (Model Context Protocol) enables:

- Tool discovery at runtime (agent can find and use new tools without restart)
- Standardized tool schemas shared across providers
- For Shipyard: MCP could expose Ship API endpoints as tools dynamically
- MVP: hardcode tools. Post-MVP: MCP tool registry for extensibility

SWE-bench Context (Architecture > Model Selection)

Key 2026 data points that validate Shipyard's architecture:

- **Opus 4.6**: 80.9% on SWE-bench Verified — highest score, proving frontier models can solve real GitHub issues
- **22-point scaffold swing**: Same model (GPT-4), basic harness = 23%, optimized harness = 45%. **Architecture matters more than model choice.**
- **Can Bölük's harness research**: 13.7-point swing just from changing edit format. Tool design is a multiplier, not an afterthought.

What this means for Shipyard:

- Our 4-tier cascade, layered verification, and structured state management are the right bets
- Investing in better tools (hooks, overlay, error classification) yields more than upgrading models
- The gap between "basic agent" and "optimized agent" is 22+ percentage points — that's our opportunity

All-Claude Model Strategy

All four tiers of the edit cascade (exact → whitespace → fuzzy → full rewrite) use Claude models via Max plan. No third-party models (Morph Fast Apply, open-source 7B, etc.). Rationale:

- Max plan is flat-rate — no cost incentive to route to cheaper models
- Single SDK, single auth, single response format — zero translation overhead
- Claude's native tool use format eliminates parsing edge cases
- Tier-4 (full rewrite) benefits from Claude's reasoning depth more than from raw speed

Multi-Agent Conflict Resolution

The original design proposed "last-write-wins for same file, manual merge for overlapping edits." This is insufficient for production use.

Real conflict scenarios:

1. Two workers edit different functions in the same file (non-overlapping) — should auto-merge
2. Two workers edit the same function (overlapping) — needs supervisor arbitration
3. Worker A adds an import, Worker B adds a different import to the same file — should auto-merge
4. Worker A refactors a function, Worker B calls the old signature — needs re-plan

Shipyard's conflict strategy:

- **Detection**: File hash comparison before/after each worker. Hash mismatch = potential conflict.
- **Non-overlapping**: Apply edits sequentially (second worker re-reads file, re-applies its edit)
- **Overlapping**: Supervisor node re-plans the conflicting step with both workers' context
- **Structural**: Type-check after merge. If errors, feed errors to the worker that introduced them.
- **MVP approach**: Sequential execution (no parallel workers initially). Conflict resolution is Phase 6.

Sources

OpenCode

- OpenCode GitHub
- How Coding Agents Actually Work: Inside OpenCode
- Inside OpenCode: How to Build an AI Coding Agent
- Building Agent Teams in OpenCode
- Context Management and Compaction

Open SWE / LangChain

- Introducing Open SWE
- Open SWE Framework
- Building Multi-Agent Applications with Deep Agents
- Deep Agents Overview
- DeepWiki: Open SWE Architecture

Claude Code

- How Claude Code Works
- Create Custom Subagents
- Text Editor Tool
- Claude Code Sandboxing
- How Claude Code is Built

File Editing Strategy

- Code Surgery: How AI Assistants Make Precise Edits
- AI File Editing Playbook
- Edit Formats -- Aider
- The Harness Problem -- Can Bölük (hashline research)
- Hashline vs Replace: Does the Edit Format Matter? -- DEV.to (counter-benchmark)
- Claude Code hashline feature request #25775 (closed NOT_PLANNED)

Provider Portability & SDK Abstraction

- OpenAI API vs Anthropic API: Developer Guide -- eesel.ai
- MCP vs Function Calling -- Portkey
- LiteLLM: Universal LLM Proxy
- AI SDK 6 -- Vercel (provider-agnostic tools)
- OpenClaw -- Wikipedia
- OpenClaw -- GitHub
- Model Context Protocol -- Anthropic (open standard)
- Agent Skills as Open Standard -- Anthropic
- Agentic AI Foundation (Linux Foundation, MCP governance)

LangGraph + Tracing

- Context Engineering for Agents
- Turn Claude Code into Domain-Specific Agent
- Trace LangGraph Applications
- LangSmith for Agent Observability